

Министерство науки и высшего образования Российской Федерации
ИТМО

Лабораторная работа №1

UNIX знакомство: useradd, nano, chmod, docker, GIT, CI, CD

по дисциплине:
"C++ и UNIX системы"

Выполнил:
Студент 3 курса, группы Z3300
Заварина Софья

Преподаватель:
Маслов И.Д.

Санкт-Петербург
2026

Цель

Познакомить студента с основами администрирования программных комплексов в ОС семейства UNIX, продемонстрировать особенности виртуализации и контейнеризации, продемонстрировать преимущества использования систем контроля версий (на примере GIT)

Задача и решение

1. [ОС] Работа в ОС, использование файловой системы, прав доступа, исполнение файлов

1.1. В папке /usr/local/ создать 2 директории: folder_max, folder_min

```
sudo mkdir /usr/local/folder_max /usr/local/folder_min
```

1.2. Создать 2-х группы пользователей: group_max, group_min

```
sudo groupadd group_max && sudo groupadd group_min
```

1.3. Создать 2-х пользователей: user_max_1, user_min_1

```
sudo useradd -m user_max_1
sudo useradd -m user_min_1
```

1.4. Для пользователей из группы *_max дать полный доступ на директории *_max и *_min. Для пользователей группы *_min дать полный доступ только на директорию *_min

```
sudo usermod -aG group_max user_max_1
sudo usermod -aG group_min user_min_1
sudo chown root:group_min /usr/local/folder_min
sudo chmod 770 /usr/local/folder_min
sudo chown root:group_max /usr/local/folder_max
sudo chmod 770 /usr/local/folder_max
sudo setfacl -m g:group_min:rwX /usr/local/folder_max
```

1.5. Создать и исполнить (пользователем из той же категории) скрипт в директории folder_max, который пишет текущую дату/время в файл output.log в текущей директории

```
sudo nano /usr/local/folder_max/script1.sh
#!/bin/bash
date >> output.log
sudo chmod +x /usr/local/folder_max/script1.sh
sudo -u user_max_1 bash -c "cd /usr/local/folder_max && ./script1.sh"
```



Рис. 1: Текущая дата

1.6. Создать и исполнить (пользователем из той же категории) скрипт в директории folder_max, который пишет текущую дату/время в файл output.log в директории *_min

```
sudo nano /usr/local/folder_max/script2.sh
#!/bin/bash
date >> /usr/local/folder_min/output.log
sudo chmod +x /usr/local/folder_max/script2.sh
sudo setfacl -m g:group_max:rwX /usr/local/folder_min
sudo -u user_max_1 bash -c "cd /usr/local/folder_max && ./script2.sh"
```



Рис. 2: Текущая дата

1.7. Исполнить (пользователем `*_min`) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в директории `*_min`

```
sudo rm -f /usr/local/folder_min/output.log
sudo -u user_min_1 bash -c "cd /usr/local/folder_max && ./script2.sh"
```



Рис. 3: Текущая дата

1.8. Создать и исполнить (пользователем из той же категории) скрипт в директории `folder_min`, который пишет текущую дату/время в файл `output.log` в директории `*_max`

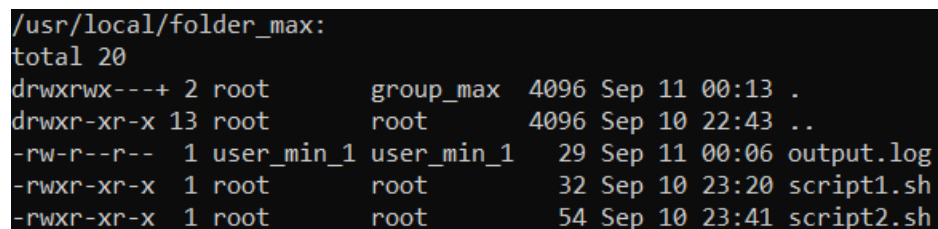
```
sudo nano /usr/local/folder_min/script3.sh
#!/bin/bash
date >> /usr/local/folder_max/output.log
sudo chmod +x /usr/local/folder_min/script3.sh
sudo rm -f /usr/local/folder_max/output.log
sudo -u user_min_1 bash -c "cd /usr/local/folder_min && ./script3.sh"
```



Рис. 4: Текущая дата

1.9. Вывести перечень прав доступа у папок `*_min/` `*_max`,

```
sudo ls -laR /usr/local/folder_max
sudo ls -laR /usr/local/folder_min
```



```
/usr/local/folder_max:
total 20
drwxrwx---+ 2 root      group_max  4096 Sep 11 00:13 .
drwxr-xr-x 13 root      root       4096 Sep 10 22:43 ..
-rw-r--r--  1 user_min_1 user_min_1   29 Sep 11 00:06 output.log
-rwxr-xr-x  1 root      root         32 Sep 10 23:20 script1.sh
-rwxr-xr-x  1 root      root         54 Sep 10 23:41 script2.sh
```

Рис. 5: Перечень прав доступа

```

/usr/local/folder_min:
total 16
drwxrwx---+ 2 root      group_min  4096 Sep 11 00:13 .
drwxr-xr-x 13 root      root        4096 Sep 10 22:43 ..
-rw-r--r--  1 user_min_1 user_min_1   29 Sep 10 23:58 output.log
-rwxr-xr-x  1 root      root         54 Sep 11 00:04 script3.sh

```

Рис. 6: Перечень прав доступа

2. [КОНТЕЙНЕР] docker build / run / ps / images

2.1. Создать скрипт, который пишет текущую дату/время в файл output.log в текущей директории

```

mkdir -p ~/lab1_docker
cd ~/lab1_docker
#!/bin/bash
date >> output.log
chmod +x script.sh

```

```

drwxr-xr-x 2 coolz coolz 4096 Sep 11 00:23 .
drwxr-x-- 7 coolz coolz 4096 Sep 11 00:22 ..
-rwxr-xr-x 1 coolz coolz  32 Sep 11 00:23 script.sh

```

Рис. 7: Текущая дата

2.2. Собрать образ со скриптами выше и с пакетом nano (docker build)

```

nano Dockerfile
FROM ubuntu:latest
RUN apt-get update && apt-get install -y nano
COPY script.sh /script.sh
RUN chmod +x /script.sh
CMD ["/script.sh"]
docker build -t my_lab1 .

```

2.3. Запустить образ (docker run)

```

docker run --rm my_lab1

```

2.4. Выполнить скрипт, который подложили при сборке образа

```

docker run --rm my_lab1 /script.sh

```

2.5. Вывести список пользователей в собранном образе

```

docker run --rm my_lab1 cat /etc/passwd

```

```

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash

```

Рис. 8: Список пользователей

3. [GIT] GitHub / GitLab, в котором будут содержаться все выполненные ЛР

3.1. Создать репозиторий в GitHub или GitLab

```

git config --global user.name "sofyayaya"
git config --global user.email "sofyayazavarina@mail.ru"
cd ~
git clone https://github.com/sofyayaya/labs.git

```

3.2. Создать структуру репозитория: 3.2.1. lab_01, 3.2.1.1. build 3.2.1.2. src 3.2.1.3. doc 3.2.1.4. cmake (для ЛР 1 опционально)

```

cd ~/labs
mkdir -p lab_01/build lab_01/src lab_01/doc lab_01/cmake

```

3.2.2. lab_02 3.2.2.1. идентично lab_01

```

mkdir -p lab_02/build lab_02/src lab_02/doc lab_02/cmake

```

3.3. Создать ветки dev / stg / prd, удалить ранее существующие ветки удаленно и локально

```

mkdir -p lab_01/{build,src,doc,cmake} lab_02/{build,src,doc,cmake}
touch lab_01/{build,src,doc,cmake}/.gitkeep
touch lab_02/{build,src,doc,cmake}/.gitkeep
git add .
git commit -m "Создана структура репозитория: lab_01, lab_02"
git checkout -b dev
git checkout -b stg
git checkout -b prd
git checkout dev
git push origin dev stg prd
git branch -D main
git push origin --delete main

```

3.4. Создать скрипт автоматического переноса ревизий из ветки dev в ветку stg с установкой метки времени (tag). Скрипт в корень репозитория

```
nano promote.sh
#!/bin/bash
set -e
git checkout stg
git merge dev --no-edit
git tag "release_${TIMESTAMP}"
git push origin stg --tags
git checkout dev
chmod +x promote.sh
git add promote.sh
git commit -m "Добавлен скрипт promote.sh для переноса dev -> stg с тегом"
git push origin dev
./promote.sh
```

3.5. Создать скрипт автоматического переноса ревизий из ветки stg в ветку prd с установкой метки времени (tag). Скрипт в корень репозитория

```
nano release.sh
#!/bin/bash
set -e
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
git checkout prd
git merge stg --no-edit
git tag "prod_release_${TIMESTAMP}"
git push origin prd --tags
git checkout dev
chmod +x release.sh
git add release.sh
git commit -m "Добавлен скрипт release.sh для переноса stg -> prd с тегом"
git push origin dev
./release.sh
```

4. [SAVE] Всё, что было сделано в шагах 1-3, сохранить в репозиторий (+ отчет по данной ЛР в папку doc). Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd.

Заключение

В ходе лабораторной работы я ознакомилась с тремя важными темами: UNIX, Docker и Git.

UNIX. Научилась создавать пользователей и группы, разграничивать права.

Docker. Освоила сборку образов и запуск контейнеров.

Git. Разобралась с ветками и тегами, научилась делать структуру репозитория и написала два скрипта — promote.sh и release.sh.

В целом работа дала понимание, как связаны система, окружение и версии кода.